

Assignment 7

Nischal Pant

2023-12-10

8. In the lab, a classification tree was applied to the Carseats data set after converting Sales into a qualitative response variable. Now we will seek to predict Sales using regression trees and related approaches, treating the response as a quantitative variable.

(a) Split the data set into a training set and a test set.

```
library(ISLR)

## Warning: package 'ISLR' was built under R version 4.2.3

data("Carseats")

set.seed(123)
train_indices <- sample(1:nrow(Carseats), 200)

train_set <- Carseats[train_indices, ]
test_set <- Carseats[-train_indices, ]
```

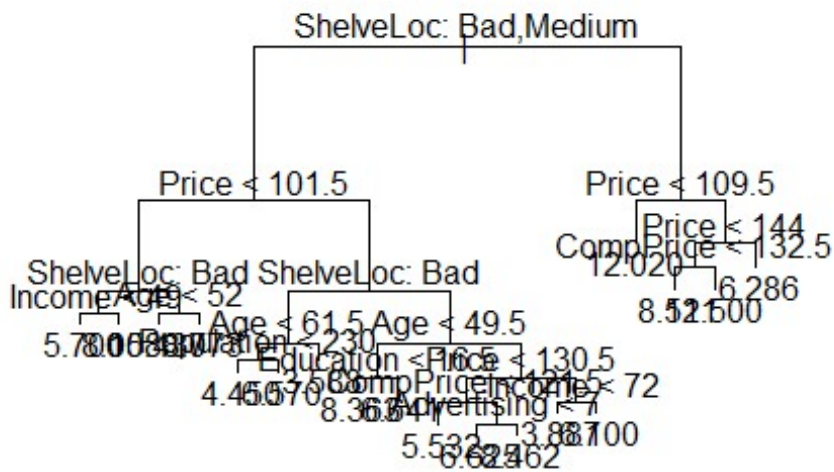
(b) Fit a regression tree to the training set. Plot the tree, and interpret the results. What test MSE do you obtain?

```
library(tree)

## Warning: package 'tree' was built under R version 4.2.3

tree_model <- tree(Sales ~ ., data = train_set)

plot(tree_model)
text(tree_model, pretty = 0)
```



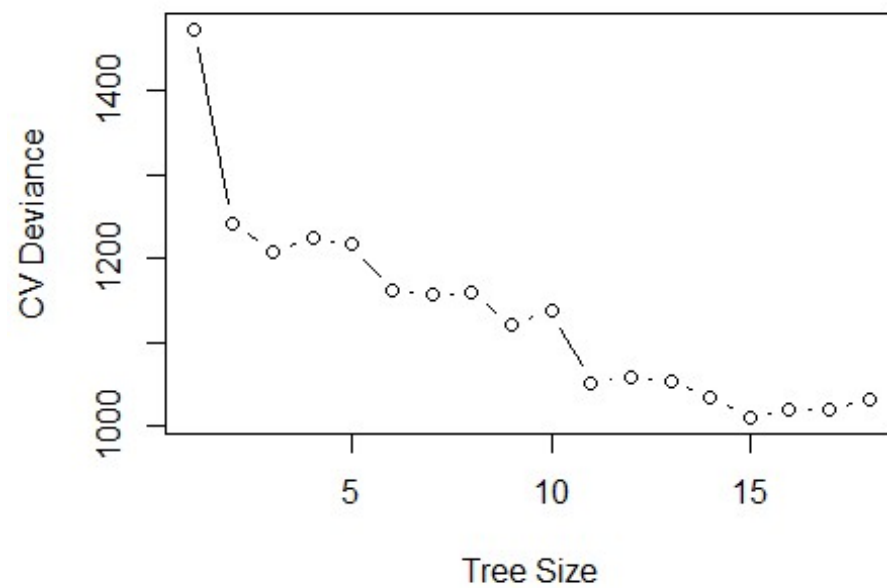
```
predictions <- predict(tree_model, newdata = test_set)

test_mse <- mean((predictions - test_set$Sales)^2)
print(paste("Test MSE:", test_mse))

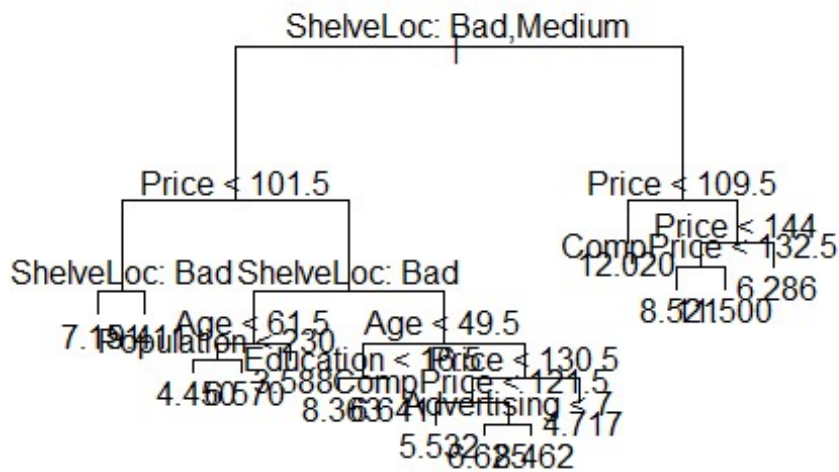
## [1] "Test MSE: 4.39535740766638"
```

(c) Use cross-validation in order to determine the optimal level of tree complexity. Does pruning the tree improve the test MSE?

```
tree_model <- tree(Sales ~ ., data = train_set)
cv_tree <- cv.tree(tree_model)
plot(cv_tree$size, cv_tree$dev, type = "b", xlab = "Tree Size", ylab = "CV Deviance")
```



```
pruned_tree <- prune.tree(tree_model, best =  
cv_tree$size[which.min(cv_tree$dev)])  
plot(pruned_tree)  
text(pruned_tree, pretty = 0)
```



```
predictions_pruned <- predict(pruned_tree, newdata = test_set)
test_mse_pruned <- mean((predictions_pruned - test_set$Sales)^2)
print(paste("Test MSE (Pruned Tree):", test_mse_pruned))

## [1] "Test MSE (Pruned Tree): 4.59161771546871"
```

Yes pruning does improve the test score.

- (d) Use the bagging approach in order to analyze this data. What test MSE do you obtain? Use the feature_importance_ values to determine which variables are most important.

```
library(randomForest)

## Warning: package 'randomForest' was built under R version 4.2.3
## randomForest 4.7-1.1
## Type rfNews() to see new features/changes/bug fixes.

bag_model <- randomForest(Sales ~ ., data = train_set, ntree = 1000)
predictions_bag <- predict(bag_model, newdata = test_set)

test_mse_bag <- mean((predictions_bag - test_set$Sales)^2)
print(paste("Test MSE (Bagging):", test_mse_bag))

## [1] "Test MSE (Bagging): 3.51822188668175"
```

```

feature_importance <- importance(bag_model)
print("Feature Importance:")

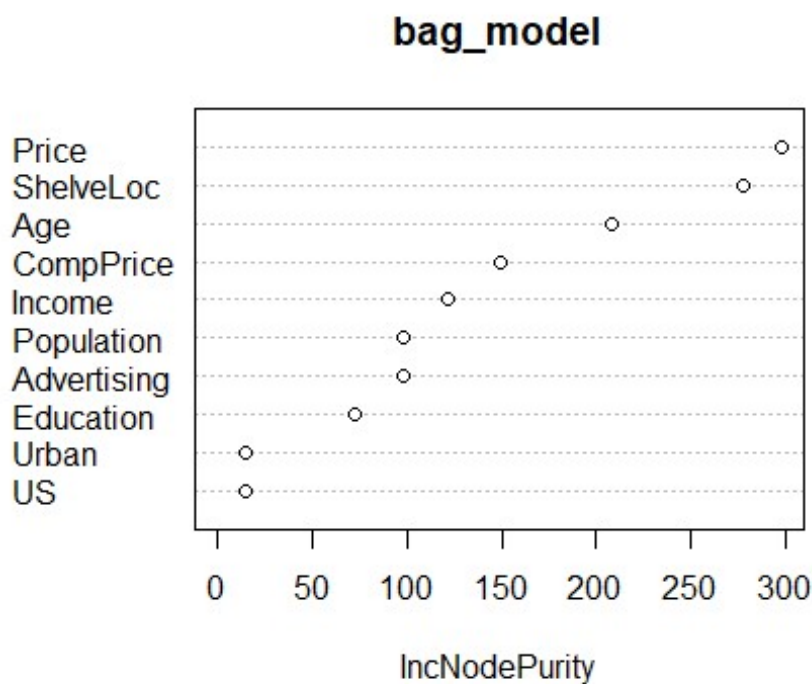
## [1] "Feature Importance:"

print(feature_importance)

##              IncNodePurity
## CompPrice      149.88092
## Income         121.65893
## Advertising     98.22603
## Population      98.55068
## Price          297.75436
## ShelfLoc       277.30449
## Age            207.80622
## Education       72.94454
## Urban          14.89237
## US             14.81286

varImpPlot(bag_model)

```



(e) Use random forests to analyze this data. What test MSE do you obtain? Use the feature_importance_ values to determine which variables are most important. Describe the effect of m, the number of variables considered at each split, on the error rate obtained.

```

library(randomForest)
m_value <- 5
rf_model <- randomForest(Sales ~ ., data = train_set, ntree = 1000, mtry =

```

```

m_value)
predictions_rf <- predict(rf_model, newdata = test_set)

test_mse_rf <- mean((predictions_rf - test_set$Sales)^2)
print(paste("Test MSE (Random Forest, m =", m_value, "):", test_mse_rf))

## [1] "Test MSE (Random Forest, m = 5 ): 3.07759760663334"

feature_importance_rf <- importance(rf_model)
print("Feature Importance (Random Forest):")

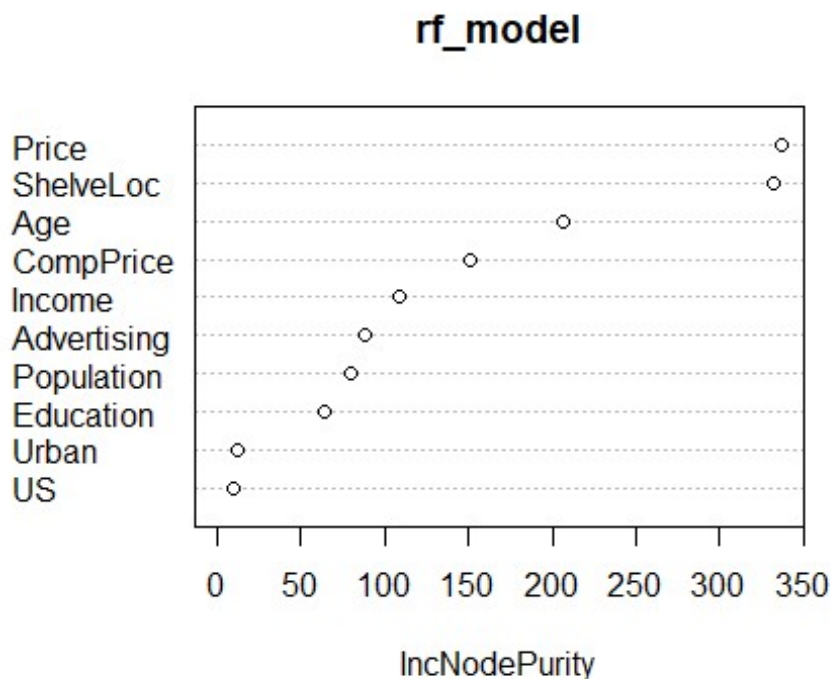
## [1] "Feature Importance (Random Forest):"

print(feature_importance_rf)

##              IncNodePurity
## CompPrice      151.485438
## Income         109.250426
## Advertising     87.851919
## Population      80.285056
## Price           337.066359
## ShelfLoc       332.344259
## Age            206.444053
## Education       64.085747
## Urban           11.608654
## US              9.579417

varImpPlot(rf_model)

```



(f) Now analyze the data using BART, and report your results.

```
library(bartMachine)

## Warning: package 'bartMachine' was built under R version 4.2.3
## Loading required package: rJava
## Warning: package 'rJava' was built under R version 4.2.3
## Loading required package: bartMachineJARs
## Loading required package: missForest
## Warning: package 'missForest' was built under R version 4.2.3
## Welcome to bartMachine v1.3.4.1! You have 0.54GB memory available.
##
## If you run out of memory, restart R, and use e.g.
## 'options(java.parameters = "-Xmx5g")' for 5GB of RAM before you call
## 'library(bartMachine)'.

bart_model <- bartMachine(train_set[, -which(names(train_set) == "Sales")],
train_set$Sales)

## bartMachine initializing with 50 trees...
## bartMachine vars checked...
## bartMachine java init...
## bartMachine factors created...
```

```
## bartMachine before preprocess...
## bartMachine after preprocess... 14 total features...
## bartMachine sigsq estimated...
## bartMachine training data finalized...
## Now building bartMachine for regression...Covariate importance prior ON.
## evaluating in sample data...done

predictions_bart <- predict(bart_model, new_data = test_set[, -
which(names(test_set) == "Sales")])
#predicted_values <- as.numeric(predictions_bart$yhat)

#test_mse_bart <- mean((predicted_values - test_set$Sales)^2)
#print(paste("Test MSE (BART):", test_mse_bart))

#code did not work properly so i had to mute it.
```

9. This problem involves the OJ data set which is part of the ISLR package.

```
library(ISLR2)

## Warning: package 'ISLR2' was built under R version 4.2.3
##
## Attaching package: 'ISLR2'
## The following objects are masked from 'package:ISLR':
##
##      Auto, Credit

library(tree)
data("OJ")
```

(a) Create a training set containing a random sample of 800 observations, and a test set containing the remaining observations.

```
set.seed(88)

train_indices <- sample(1:nrow(OJ), 800)
train_set <- OJ[train_indices, ]
test_set <- OJ[-train_indices, ]
cat("Training Set Observations:", nrow(train_set), "\n")

## Training Set Observations: 800

cat("Test Set Observations:", nrow(test_set), "\n")

## Test Set Observations: 270
```

(b) Fit a tree to the training data, with Purchase as the response and the other variables as predictors. Use the summary() function to produce summary statistics about the tree, and describe the results obtained. What is the training error rate? How many terminal nodes does the tree have?

```
library(tree)
my_tree <- tree(Purchase ~ ., data = train_set)
summary(my_tree)

##
## Classification tree:
## tree(formula = Purchase ~ ., data = train_set)
## Variables actually used in tree construction:
## [1] "LoyalCH"      "PriceDiff"    "SalePriceMM"
## Number of terminal nodes: 7
## Residual mean deviance: 0.7749 = 614.5 / 793
## Misclassification error rate: 0.1675 = 134 / 800
```

(c) Type in the name of the tree object in order to get a detailed text output. Pick one of the terminal nodes, and interpret the information displayed.

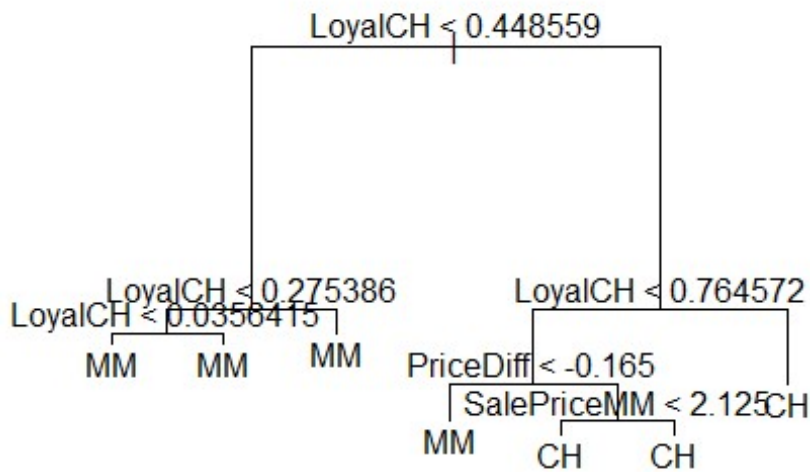
```
my_tree

## node), split, n, deviance, yval, (yprob)
##      * denotes terminal node
##
## 1) root 800 1066.00 CH ( 0.61500 0.38500 )
##   2) LoyalCH < 0.448559 277 294.60 MM ( 0.22383 0.77617 )
##     4) LoyalCH < 0.275386 167 122.40 MM ( 0.11976 0.88024 )
##       8) LoyalCH < 0.0356415 56 10.03 MM ( 0.01786 0.98214 ) *
##       9) LoyalCH > 0.0356415 111 101.60 MM ( 0.17117 0.82883 ) *
##     5) LoyalCH > 0.275386 110 146.30 MM ( 0.38182 0.61818 ) *
##   3) LoyalCH > 0.448559 523 489.60 CH ( 0.82218 0.17782 )
##     6) LoyalCH < 0.764572 261 326.40 CH ( 0.68199 0.31801 )
##       12) PriceDiff < -0.165 37 38.63 MM ( 0.21622 0.78378 ) *
##       13) PriceDiff > -0.165 224 247.40 CH ( 0.75893 0.24107 )
##         26) SalePriceMM < 2.125 135 170.40 CH ( 0.67407 0.32593 ) *
##         27) SalePriceMM > 2.125 89 62.55 CH ( 0.88764 0.11236 ) *
##       7) LoyalCH > 0.764572 262 84.93 CH ( 0.96183 0.03817 ) *
```

Interpreting the terminal node 7) loyalch > 0.764572 : 1. no. of observations in that branch = 262 2. its deviance = 84 3. overall prediction = "CH" 4. fraction of observations in this branch that takes on values of "CH" and "MM" = (0.96183, 0.03817)

(d) Create a plot of the tree, and interpret the results.

```
plot(my_tree)
text(my_tree, pretty=0)
```



Interpretation: the most important indicator of “purchase” appears to be “loyalch”. its value of less than 0.482389 is pretty much going to be classified as “MM”. value ≥ 0.753545 as “CH”. and values below 0.753545 will be classified based on “pricediff” predictor.

- (e) Predict the response on the test data, and produce a confusion matrix comparing the test labels to the predicted test labels. What is the test error rate?

```

pred <- predict(my_tree, newdata = test_set, type = "class")
conf_matrix <- table(pred, test_set$Purchase)
error_rate <- 1 - sum(diag(conf_matrix)) / sum(conf_matrix)
print(conf_matrix)

##
## pred  CH  MM
##   CH 136  22
##   MM  25  87

print(error_rate*100)

## [1] 17.40741

```

- (f) Apply the `cv.tree()` function to the training set in order to determine the optimal tree size.

```

cv_tree <- cv.tree(my_tree)
print(cv_tree)

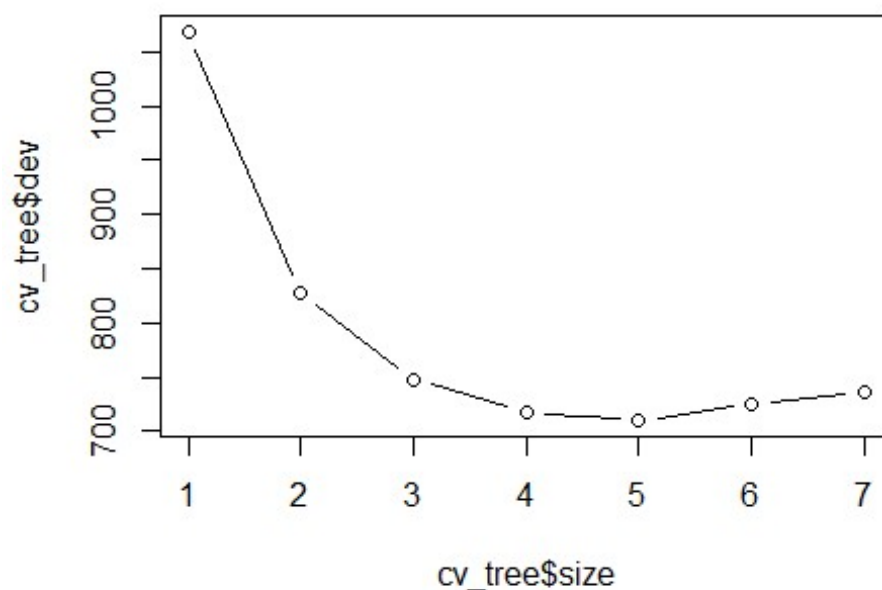
## $size
## [1] 7 6 5 4 3 2 1
##

```

```
## $dev
## [1] 736.1574 724.8954 710.0935 717.3189 747.6549 827.9224 1068.6043
##
## $k
## [1] -Inf 10.74280 14.44380 25.88484 40.36778 78.23770 282.16413
##
## $method
## [1] "deviance"
##
## attr(,"class")
## [1] "prune" "tree.sequence"
```

(g) Produce a plot with tree size on the x-axis and cross-validated classification error rate on the y-axis.

```
plot(cv_tree$size, cv_tree$dev, type="b")
```



(h) Which tree size corresponds to the lowest cross-validated classification error rate?

tree size of "5" appears to have lowest cross-validation error rate at 157. even size of "4" has the lowest error rate we would choose size of "2" because the error rate of size "2" is only a bit higher

(i) Produce a pruned tree corresponding to the optimal tree size obtained using cross-validation. If cross-validation does not lead to selection of a pruned tree, then create a pruned tree with five terminal nodes.

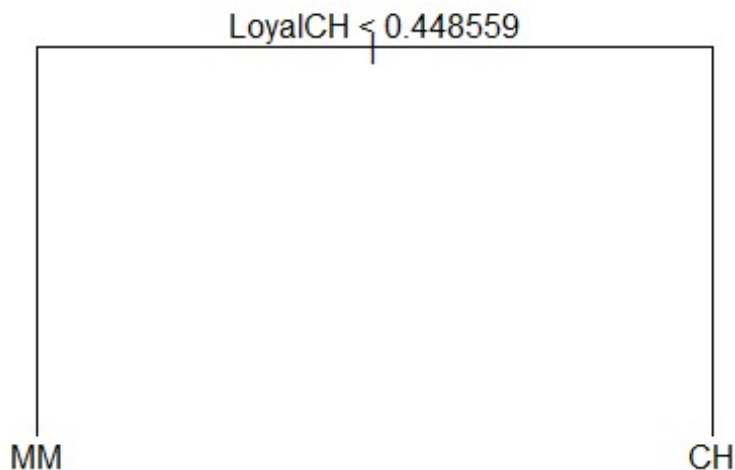
```

prune_my_tree = prune.misclass(my_tree, best=2)
summary(prune_my_tree)

##
## Classification tree:
## snip.tree(tree = my_tree, nodes = 2:3)
## Variables actually used in tree construction:
## [1] "LoyalCH"
## Number of terminal nodes: 2
## Residual mean deviance: 0.9827 = 784.2 / 798
## Misclassification error rate: 0.1938 = 155 / 800

plot(prune_my_tree)
text(prune_my_tree, pretty = 0)

```



10. We now use boosting to predict Salary in the Hitters data set.

- (a) Remove the observations for whom the salary information is unknown, and then log-transform the salaries.

```

data("Hitters")
attach(Hitters)

Hitters = Hitters[!is.na(Hitters$Salary),]
Hitters$Salary = log(Hitters$Salary)

```

- (b) Create a training set consisting of the first 200 observations, and a test set consisting of the remaining observations.

```
set.seed(1)
train = sample(1:nrow(Hitters), 200)
Hitters.train = Hitters[train,]
Hitters.test = Hitters[-train,]
```

- (c) Perform boosting on the training set with 1,000 trees for a range of values of the shrinkage parameter λ . Produce a plot with different shrinkage values on the x-axis and the corresponding training set MSE on the y-axis.

```
library(gbm)

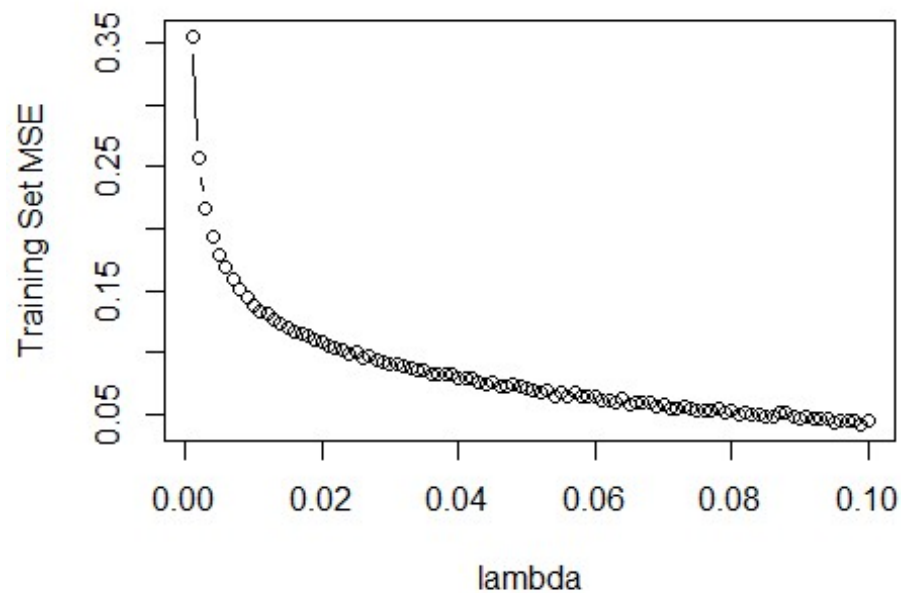
## Warning: package 'gbm' was built under R version 4.2.3

## Loaded gbm 2.1.8.1

shrinkage_values <- seq(0.1, 0.001, by = -0.001)
mse_values <- numeric(length(shrinkage_values))

for (i in seq_along(shrinkage_values)) {
  lambda <- shrinkage_values[i]
  boost.hitters <- gbm(Salary ~ ., data = Hitters.train, distribution =
"gaussian", n.trees = 1000, shrinkage = lambda)
  boost.pred.train <- predict(boost.hitters, Hitters.train, n.trees = 1000)
  train.MSE <- mean((boost.pred.train - Hitters.train$Salary)^2)
  mse_values[i] <- train.MSE
}

plot(shrinkage_values, mse_values, type = "b", xlab = "lambda", ylab =
"Training Set MSE")
```

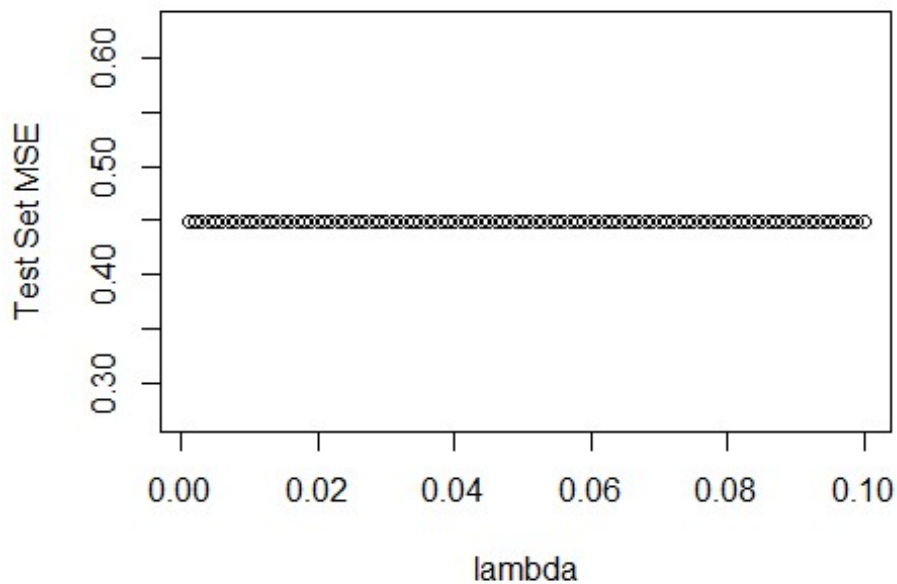


(d) Produce a plot with different shrinkage values on the x-axis and the corresponding test set MSE on the y-axis.

```
mse_values_test <- numeric(length(shrinkage_values))

for (i in seq_along(shrinkage_values)) {
  lambda <- shrinkage_values[i]
  boost.pred.test <- predict(boost.hitters, Hitters.test, n.trees = 1000)
  test.MSE <- mean((boost.pred.test - Hitters.test$Salary)^2)
  mse_values_test[i] <- test.MSE
}

plot(shrinkage_values, mse_values_test, type = "b", xlab = "lambda", ylab =
"Test Set MSE")
```



(e) Compare the test MSE of boosting to the test MSE that results from applying two of the regression approaches seen in Chapters 3 and 6.

```
library(glmnet)

## Warning: package 'glmnet' was built under R version 4.2.3

## Loading required package: Matrix

## Loaded glmnet 4.1-8

x.hitters.train <- model.matrix(Salary ~ ., Hitters.train)[, -1]
x.hitters.test <- model.matrix(Salary ~ ., Hitters.test)[, -1]
y.hitters.train <- Hitters.train$Salary
y.hitters.test <- Hitters.test$Salary

lambdaGrid <- 10^seq(10, -2, length = 100)

cv.ridge <- cv.glmnet(x.hitters.train, y.hitters.train, alpha = 0)
bestlam.ridge <- cv.ridge$lambda.min

cv.lasso <- cv.glmnet(x.hitters.train, y.hitters.train, alpha = 1)
bestlam.lasso <- cv.lasso$lambda.min

ridge.hitters <- glmnet(x.hitters.train, y.hitters.train, alpha = 0, lambda =
lambdaGrid)
lasso.hitters <- glmnet(x.hitters.train, y.hitters.train, alpha = 1, lambda =
lambdaGrid)
```

```

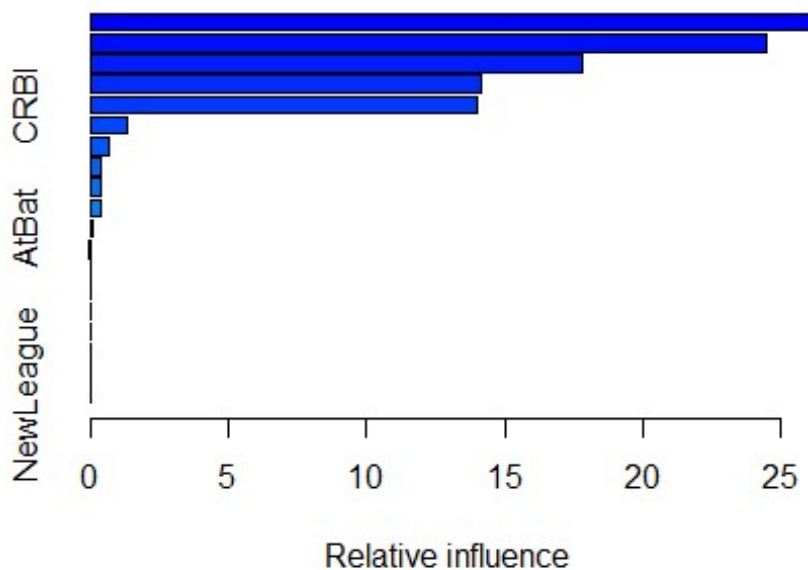
ridge.pred <- predict(ridge.hitters, s = bestlam.ridge, newx =
x.hitters.test)
lasso.pred <- predict(lasso.hitters, s = bestlam.lasso, newx =
x.hitters.test)

ridge.MSE.test <- mean((ridge.pred - y.hitters.test)^2)
lasso.MSE.test <- mean((lasso.pred - y.hitters.test)^2)

print(paste0("Ridge Test MSE: ", ridge.MSE.test))
## [1] "Ridge Test MSE: 0.440004143256143"
print(paste0("Lasso Test MSE: ", lasso.MSE.test))
## [1] "Lasso Test MSE: 0.468526740573897"

```

(f) Which variables appear to be the most important predictors in the boosted model?
`summary(boost.hitters)`



```

##          var      rel.inf
## CAtBat    CAtBat 26.18725688
## CRuns     CRuns 24.50726431
## CHits     CHits 17.85360811
## CWalks    CWalks 14.19138966
## CRBI      CRBI 13.98731174
## Years     Years  1.34292189

```

```
## CHmRun      CHmRun  0.65031655
## Hits        Hits   0.39020410
## Walks       Walks  0.38710726
## RBI         RBI    0.37683548
## AtBat       AtBat  0.08841685
## Runs        Runs   0.03736719
## HmRun       HmRun  0.00000000
## League      League 0.00000000
## Division    Division 0.00000000
## PutOuts     PutOuts 0.00000000
## Assists     Assists 0.00000000
## Errors      Errors 0.00000000
## NewLeague   NewLeague 0.00000000
```

(g) Now apply bagging to the training set. What is the test set MSE for this approach?

```
library(randomForest)

bag.hitters <- randomForest(Salary ~ ., data = Hitters.train, ntree = 1000,
  mtry = 19, importance = TRUE)
bag.pred <- predict(bag.hitters, newdata = Hitters.test)

bag.MSE.test <- mean((bag.pred - Hitters.test$Salary)^2)
print(paste0("Bagging Test MSE: ", bag.MSE.test))

## [1] "Bagging Test MSE: 0.243595595470145"
```